Shipping-document extraction: zero-shot VLM + second-reader review flag

Repo: https://github.com/romitheguru/shipdoc_info_extraction · test predictions: `test_predictions.json` · notes behind every decision: `analysis-notes/01–12` · No HF model / W&B run: nothing is trained.

1. Approach and why

```
pages → classify p1 → Qwen3-VL-8B (4-bit, MLX), per-type prompt, all pages in one call → column roles + 2 repairs → normalise
                                                  PaddleOCR-VL reads each page → "value not on the page" → review flags
```

- **Zero-shot VLM, not OCR + rules.** In a 40-doc bake-off covering all 18 suppliers (note 09), Qwen3-VL-8B read supplier part numbers exactly 98.9% (digital) / 92.3% (scanned) of the time, above the *ceiling* of PP-OCRv5 (95.1 / 86.0: a value OCR never emits can't be extracted). It got 100% of header fields and 0 of 15 false fills on illegible fields with no rules at all.
- **No fine-tuning.** Train has 18 suppliers whose ~40-part catalogues are reused in 99% of dev rows (note 02), so a tuned model would learn to recall catalogues, which is exactly what fails on unseen suppliers. It would also cost a GPU we chose not to rent.
- **Field meaning from the model, formats from code.** The invoice prompt asks the model to name each column's *role* from the seller's point of view ("Your/Customer" = buyer) before it fills rows. This targets the costliest error (one swapped column zeroes every row). Code then does dates (ISO, ambiguous day/month flagged), numbers (incl. European `1.234,50`), currency and IATA codes. No rule mentions a supplier, prefix or layout position.
- **Sized for 24 GB.** Size sweep (note 11): 2B 55.5 OVERALL with false fills, 4B 84.0, 8B 4-bit 90.5, 8B 8-bit 89.4 (40-doc bake-off). With the final prompt, **8B 4-bit** scored 91.0 vs 90.3 for 8-bit, with 0 vs 1 false fill, at 19 vs 31 s/doc and a 7.5 vs 11.9 GB peak. We kept 4-bit, which leaves about two-thirds of the 24 GB free. The quantisation risk is real but not decisive: both widths read `P0` as `P0` on one invoice.
- **Null only when obscured.** Null on a readable field scores like a wrong value (note 07), so readable-but-uncertain values are kept and *flagged*; only absent, blacked-out or scribbled values are null.

2. Dev results (100 docs; official scorer, identical numbers from `eval.evaluate`)

slice	docs	OVERALL	header %	row F1	docs exact %	false fills
all	100	**92.5**	99.9	94.0	75.0	1/34
scanned	43	**91.7**	100.0	92.2	74.4	0/12
digital	57	**93.0**	99.8	95.1	75.4	1/22
multi-page	28	**83.9**	100.0	93.6	32.1	0/9
repeated parts	17	**89.9**	100.0	89.5	70.6	0/5
illegible field	8	**87.5**	98.4	89.1	62.5	1/12
waybills	23	**100.0**	100.0	–	100.0	0/4
invoices	77	**91.0**	99.8	94.0	67.5	1/30
unseen: fold 4, clean (no prompt change from these)	22	**87.1**	100.0	90.6	54.5	0/10
seen: other folds (prompt iterated on errors here)	78	**94.3**	99.8	95.5	80.8	1/24

3. Failure modes

25 of 100 dev docs are not fully correct. Header fields are 99.9% right, and each header error is a false fill, so almost all lost points are **rows**: 55 of 924 predicted rows are wrong, and each wrong row has exactly one wrong field.

failure mode	rows	docs	example (predicted → gold)	doc flagged?
Lookalike characters in part numbers: S→8/5, O↔0, Z→2, I↔1/L, P→F	38	18	`NGTT-1750-86` → `…-S6`, `LH-25860-120U` → `…120U`	16 of 18
Systematic prefix misread on one digital invoice (P0 read as `P0`, fold 4)	9	1	`P04503319053` → `PO4503319053`	yes
Dropped, added or garbled characters, incl. one Cyrillic `М` emitted for Latin M	7	6	`GHSR20581W` → `GHSR2058I1W`	6 of 6
Missing cell (quantity read as null)	1	1	dev_0093	no
False fill: a scribbled-out date was read as a date	1 field	1	dev_0084: 2024-09-24 for an illegible date	yes (date not found by the second reader)

- **Column meaning is solved on dev:** 0 column swaps. The 2 swaps in the original bake-off prompt (35 rows) are fixed by the column-role prompt and the two generic repairs.
- **Unseen vs seen.** Fold 4 scores 87.1 against 94.3 for the other folds. The gap is character reading, not layout: fold 4's part-number styles (`GLT/…/S5S`, `0VVX…S`, `JI-…00FN`) are dense with S/5/8 and O/0, and it contains the `P0` invoice. No fold-4 doc has a column or header error.
- **Multi-page docs** score lowest on exactness (32% fully correct): they have the most rows, so one lookalike anywhere fails the document. Their row F1 (93.6) is close to the average.
- **Cheap next fixes** (not applied, config frozen): map non-Latin homoglyphs to Latin, then re-read flagged part numbers as crops at 2× resolution with the same model. Both are generic.

4. A new supplier layout

- **Nothing to retrain.** The prompt describes field *meanings* (seller vs buyer part number, PO even when it looks like a part number, totals only from the labelled total), not layouts; header wordings appear only as examples. A new layout goes through as-is, and fold 4 measures exactly this.
- **Monitor, then fix generically.** Review-flag rates per supplier show a new layout that reads badly. A fix is a meaning rule or a repair that holds for any layout (as the two existing column repairs do), checked on a held-out fold before it ships, never a supplier-specific rule.
- **Volume.** At ~10k docs/day this design moves to a 24 GB GPU with vLLM batching and JSON-schema-constrained decoding (note 10); that path is a recommendation, untested here.

5. Needs-review flag (separate file; never changes a value)

A value is flagged when the independent reader (PaddleOCR-VL) does not find it on the page, or when a slash date could be day- or month-first; a whole document is flagged if the output was truncated. On dev: **document level** precision 68.8%, recall 88.0% (a flagged doc is not fully correct); **value level** precision 28.0%, recall 82.1% (header fields and predicted rows). Column-mapping errors are invisible to a text check by design, so they are only caught at document level when another value is also flagged.

```
doc    precision  68.8%  recall  88.0%  (flagged 32, errors 25, caught 22)
value  precision  28.0%  recall  82.1%  (flagged 164, errors 56, caught 46)
  not_in_ocr                              flags  164  precision  28.0%  errors caught  46 (only by this signal 46)
```

6. Limits and honesty

- I looked at all 18 train suppliers while analysing the data, and the column-role prompt and its three fixes were written after reading bake-off errors (fold-4 docs excluded). Dev numbers are therefore optimistic. The fold-4 row is the cleanest unseen estimate, but it has only 22 docs.
- Throughput on the Mac: over 300 docs, 20 s/doc median, 83 s p95, 107 docs/hour on one Mac, 8.1 GB peak for the extractor (+ PaddleOCR-VL ~3 s/page, 2.3 GB, run separately). The transformers backend (`--backend hf`) for graders without a Mac is implemented but was not run.
- Two test invoices (30 rows each) hit the 4,096-token output limit, and one came out empty. The limit was raised to 8,192 and only those two were re-run. No dev doc reached the old limit, so the dev numbers are unaffected.
- Cut for time: a faster prompt (v2, compact output) that turned out slower and was not used; scan upscaling; a second OCR tier for high-precision flags.